Abstract / Projektreflexion: Entwicklung des Python-basierten „Rezept_Empfehlung"

Projekt: RezeptFinder

Modulname: Projekt Software Engineering (DLMCSPSE01_D)
Name: Daniel Faller
Matrikelnummer: 10807727
Abgabedatum: 14.10.2026

1. Einleitung und Motivation

Im Rahmen des Moduls Software Engineering habe ich die Anwendung „Rezept_Empfehlung" entwickelt. Ausgangspunkt war eine alltägliche Situation: Wer nicht weiß, was er kochen soll, verliert viel Zeit mit der Suche in Kochbüchern oder im Internet. Bestehende Online-Lösungen sind oft überladen und bieten zu viele Optionen, anstatt schnell einen konkreten Vorschlag zu liefern. Die Anwendung löst dieses Problem durch eine bewusst einfach gehaltene, geführte Auswahl: Der Nutzer gibt zwei Kriterien an – ob das Gericht Fleisch enthalten soll und wie viel Zeit zur Verfügung steht – und erhält daraufhin einen passenden Rezeptvorschlag. Das Projekt verfolgte dabei zwei übergeordnete Ziele: erstens die Entwicklung einer funktionsfähigen Desktop-Anwendung mit grafischer Benutzeroberfläche, und zweitens die praktische Anwendung von Prinzipien der objektorientierten Programmierung (OOP), der strukturierten Softwaredokumentation sowie der Versionsverwaltung mit Git.

2. Anforderungsanalyse und Zielsetzung

In der Konzeptionsphase wurden die Anforderungen systematisch erfasst und priorisiert. Funktionale Anforderungen umfassen die Filterung von Rezepten über Radiobuttons nach Kategorie (Fleisch / vegetarisch) und Zubereitungszeit sowie die Anzeige eines zufällig ausgewählten, passenden Rezepts nach Bestätigung durch den Nutzer. Zu den nicht-funktionalen Anforderungen zählten Stabilität und Fehlertoleranz: Vergisst der Nutzer eine Auswahl zu treffen, soll das Programm dies sauber abfangen, anstatt mit einer unkontrollierten Fehlermeldung abzubrechen. Darüber hinaus war eine klare Klassenstruktur gefordert, die eine spätere Erweiterung um neue Rezepte oder Filterkategorien ohne größeren Aufwand ermöglicht. Die Benutzeroberfläche sollte intuitiv bedienbar sein und keine Einarbeitung erfordern. Als Entwicklungsumgebung wurde Visual Studio Code gewählt; die Dokumentation der Anforderungen erfolgte in einem separaten Anforderungsdokument mit User Stories.

3. Technische Umsetzung und Systemarchitektur

Als Programmiersprache wurde Python gewählt, da es leicht lesbar ist und mit Tkinter eine integrierte Bibliothek für grafische Oberflächen mitbringt — ohne externe Abhängigkeiten. Das Programm ist konsequent nach dem Prinzip der Trennung von Verantwortlichkeiten in drei Klassen aufgeteilt:

- **Rezept:** Datenhaltung für Name, Kategorie, Zubereitungszeit und Rezepttext. Jedes Gericht wird als eigenständiges Objekt dieser Klasse instanziiert.
- **RezeptFinder:** Verwaltung aller Rezepte in einer Liste sowie die Filterlogik; nutzt das Modul random zur zufälligen Auswahl eines passenden Eintrags aus den gefilterten Ergebnissen.
- **RezeptApp:** Aufbau des Tkinter-Fensters und Verbindung der Nutzerinteraktion mit der Hintergrundlogik. Diese Klasse fungiert als Controller zwischen GUI und Datenschicht.

Die Aufteilung in separate Klassen folgt dem Single-Responsibility-Prinzip und erleichtert die Wartbarkeit des Codes. Der Quellcode wurde mit Git versioniert und in einem öffentlichen GitHub-Repository abgelegt, sodass der gesamte Entwicklungsverlauf nachvollziehbar dokumentiert ist. UML-Diagramme (Klassendiagramm, Aktivitätsdiagramm und Sequenzdiagramm) wurden mit dem Online-Tool Draw.io erstellt und dokumentieren die Architektur sowie den Programmablauf.

4. Testphase und Testergebnisse

Das Programm wurde in der Finalisierungsphase manuell mit verschiedenen Eingabekombinationen getestet. Geprüft wurde dabei, ob die Filterung nach Kategorie und Zeit in allen möglichen Kombinationen korrekte Ergebnisse liefert, ob Fehleingaben (z. B. keine Auswahl getroffen) stabil abgefangen werden und ob die Oberfläche sich in allen Fällen erwartungsgemäß verhält. Für jede Kombination aus Fleisch/vegetarisch und kurzer/langer Zubereitungszeit wurde ein eigener Testfall definiert und dokumentiert. Zusätzlich wurde ein Grenzfall getestet, bei dem keine passenden Rezepte in der Datenbank vorhanden sind. Die Testergebnisse wurden in einem strukturierten Testdokument festgehalten. Alle Testfälle wurden erfolgreich bestanden: Das Fenster öffnet sich fehlerfrei, die Filterlogik liefert korrekte Ergebnisse und das Programm reagiert auch auf unvollständige Eingaben ohne Absturz.

5. Fazit und persönliche Reflexion

Das Projekt hat mir geholfen, zentrale Konzepte der Softwareentwicklung in der Praxis umzusetzen und miteinander zu verknüpfen. Die größte Herausforderung war die saubere Trennung von Logik und Oberfläche sowie der Einstieg in Tkinter, da ich zuvor noch keine eigene GUI entwickelt hatte. Auch die konsequente Nutzung von Git als Versionsverwaltungssystem war anfangs ungewohnt, hat sich aber als sehr hilfreich erwiesen, um Änderungen nachvollziehbar zu halten. Durch einen strukturierten Ansatz – erst die Datenklassen, dann die Filterlogik, anschließend die GUI und zuletzt die Tests – konnte ich die Anwendung schrittweise aufbauen und auftretende Probleme gezielt lösen. Rückblickend würde ich beim nächsten Projekt früher mit dem Schreiben automatisierter Unit-Tests beginnen und die Anforderungen noch detaillierter und messbarer dokumentieren. Insgesamt war das Projekt eine wertvolle Übung in objektorientierter Programmierung, Versionsverwaltung mit Git und strukturierter Softwaredokumentation — und hat mir gezeigt, wie wichtig eine durchdachte Planung vor dem eigentlichen Coden ist.